UJIAN TENGAH SEMESTER

PEMROGRAMAN BERORIENTASI OBJEK

Nama	**:**	**Jihan Maharani**
NIM	**:**	**250660221031**
Program Studi	**:**	**Sistem Informasi**
Kelas	**:**	**SI-3A**

SOAL

Bagian A — Teori

1. **Konsep Dasar OOP**
   Jelaskan pengertian dari kelima konsep berikut dan berikan satu contoh penerapannya pada kode Pyhton:
   1. Class
   2. Object
   3. Attribute
   4. Method
   5. Constructor

   Jawaban:

   1. Class
      Class adalah cetak biru (blueprint) atau template dasar yang digunakan untuk menciptakan sebuah objek.
      Contohnya:
      ```
      class Mahasiswa:
          nama = "Alfarishi"
          nim = "250660221031"
      ```

   2. Object adalah hasil atau instance dari sebuah class. Setiap object dapat memiliki nilai attribute yang berbeda.
      Contohnya:
      ```
      mahasiswa1 = Mahasiswa()
      print(mahasiswa1.nama)
      ```

   3. Attribute adalah variabel yang terdapat di dalam class dan digunakan untuk menyimpan data atau karakteristik suatu object.
      Contohnya:
      ```
      class Mahasiswa:
          def __init__(self, nama):
              self.nama = nama
      ```

4. Method adalah fungsi yang berada di dalam class dan digunakan untuk menjalankan tindakan atau perilaku suatu object.
   Contohnya:
   ```
   class Mahasiswa:
       def belajar(self):
           print("Mahasiswa sedang belajar")
   ```

5. Constructor adalah method khusus __init__() yang otomatis dijalankan ketika object dibuat. Constructor digunakan untuk memberikan nilai awal pada attribute.
   ```
   class Mahasiswa:
       def __init__(self, nama):
           self.nama = nama

   mahasiswa1 = Mahasiswa("Alfarishi")
   ```

2. Pilar OOP

Jelaskan keempat pilar OOP berikut beserta penggunaannya pada program Phyton:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

Untuk setiap pilar sertakan satu contoh kode singkat yang menunjukan penerapannya.

Jawaban:

1. Encapsulation adalah konsep membungkus data dan method dalam satu class serta membatasi akses langsung terhadap attribute. Dalam Python, attribute privat dapat ditulis menggunakan dua garis bawah (__).
   Contohnya:
   ```
   class Mahasiswa:
       def __init__(self, nama):
           self.__nama = nama

       @property
       def nama(self):
           return self.__nama

   mahasiswa1 = Mahasiswa("Alfarishi")
   print(mahasiswa1.nama)
   ```

   Penjelasan: Attribute __nama dibuat privat sehingga akses terhadapnya dilakukan melalui property nama.

2. Inheritance adalah konsep pewarisan attribute dan method dari class induk (parent) kepada class turunan (child). Konsep ini membantu penggunaan kembali kode.
   Contohnya:
   ```
   class Hewan:
       def makan(self):
           print("Hewan sedang makan")

   class Kucing(Hewan):
       pass

   kucing1 = Kucing()
   kucing1.makan()
   ```

   Penjelasan: Class Kucing mewarisi method makan() dari class Hewan.

3. Polymorphism adalah kemampuan beberapa object yang berbeda untuk menggunakan method dengan nama yang sama, tetapi memiliki perilaku atau hasil yang berbeda.
   Contohnya:
   ```
   class Kucing:
       def suara(self):
           print("Meong")

   class Anjing:
       def suara(self):
           print("Guk guk")

   hewan = [Kucing(), Anjing()]

   for item in hewan:
       item.suara()
   ```

   Penjelasan: Kedua class memiliki method suara(), tetapi menghasilkan output yang berbeda.

4. Abstraction adalah konsep menyembunyikan detail implementasi dan hanya menampilkan fungsi atau fitur penting kepada pengguna. Dalam Python, abstraction dapat diterapkan menggunakan ABC dan abstractmethod.
   Contohnya:
   ```
   from abc import ABC, abstractmethod

   class Kendaraan(ABC):
   ```

```
    @abstractmethod
    def berjalan(self):
        pass

class Mobil(Kendaraan):
    def berjalan(self):
        print("Mobil sedang berjalan")

mobil1 = Mobil()
mobil1.berjalan()
```

Penjelasan: Class Kendaraan memiliki method abstrak berjalan() yang wajib diimplementasikan oleh class turunannya.

3. Analisis Desain

Perhatikan rancangan class berikut:

```
class Pengguna:
    def __init__(self, nama, email):
        self.__nama = nama
        self.__email = email

    @property
    def nama(self):
        return self.__nama

class Mahasiswa(Pengguna):
    def __init__(self, nama, email, nim):
        super().__init__(nama, email)
        self.nim = nim

    def tampilkan_peran(self):
        return f"{self.nama} adalah mahasiswa dengan NIM {self.nim}"
```

Jawab pertanyaan berikut:

1. Konsep OOP mana yang diterapkan pada self.__nama dan @property? Jelaskan alasan teknisnya.
2. Konsep OOP mana yang diterapkan pada class Mahasiswa(Pengguna)? Jelaskan apa yang dilakukan super().
3. Mengapa self.nama dapat dipanggil pada method tampilkan_peran() di subclass meskipun attribute tersebut privat di superclass?

Jawaban:

1. Konsep OOP pada self.__nama dan @property yang diterapkan adalah encapsulation
   Alasannya:
   - self.__nama merupakan attribute privat yang membatasi akses langsung dari luar class.
   - @property digunakan untuk menyediakan akses terhadap attribute privat melalui method yang diperlakukan seperti attribute.
   - Dengan demikian, data dapat diakses tanpa memberikan akses perubahan secara langsung.

2. Konsep OOP pada class Mahasiswa(Pengguna) dan fungsi super () adalah inheritance atau pewarisan.
   Class Mahasiswa merupakan subclass dari class Pengguna, sehingga dapat mewarisi method dan attribute yang dimiliki parent class.
   Fungsi super() digunakan untuk memanggil constructor atau method dari parent class.
   Pada kode tersebut:
   super().__init__(nama, email)
   Digunakan untuk menjalankan constructor Pengguna agar attribute nama dan email diinisialisasi tanpa harus menuliskan ulang kode constructor parent.

3. Mengapa self.nama dapat dipanggil pada subclass? Karena class Mahasiswa mewarisi class Pengguna dan memiliki akses terhadap property nama yang diwariskan dari parent class.
   Meskipun attribute __nama bersifat privat, property nama dapat mengaksesnya melalui method getter yang berada di dalam class Pengguna.
   Oleh karena itu, subclass dapat memanggil self.nama tanpa mengakses attribute privat __nama secara langsung.